DOCUMENTATION LINKS

- Setup Structure
- Actions
- Basic Selectors
- Advanced Selectors
- Handlers
- The Describe json
- Describe setups
- Players <-> Spectators

CORE PRINCIPLES

WHAT IS A JSON?

If you don't know what the json file format is, here is a video explaining all you need to know.

You will write your game in a json file. This json file gets sent to the Ludio engine, which understands how to translate your game json into a playable game that runs automatically in a Ludio video call.

BUILDING BLOCKS: ACTIONS

In other platforms, *game components* are usually the building blocks of your game. You create decks of cards, dice, tokens, meeples, a board, etc. and then interact with them using a 3D simulator or physics engine. This paradigm works well for accurately modeling a physical game, but NOT well for automating and enforcing the rules.

In Ludio, **actions** (or more academically, ludemes) are the building blocks of your game. Actions model the mechanics players use to interact with the game. An action's visual representation *might* involve physical components (e.g. moving a card from one deck to another), but often it will not (e.g. voting at a player in Ludio simply requires clicking on someone else's camera).

For example, many physical games will include chips, tokens, and notepads to help players keep track of secret information, player scores, etc. Ludio dispenses with all of these "recordkeeping" components because Ludio models game mechanics, not game components.

An action has **inputs** and **outputs**. Every action takes a **payload**, which is a collection of all its inputs; and a **result**, which is its output.

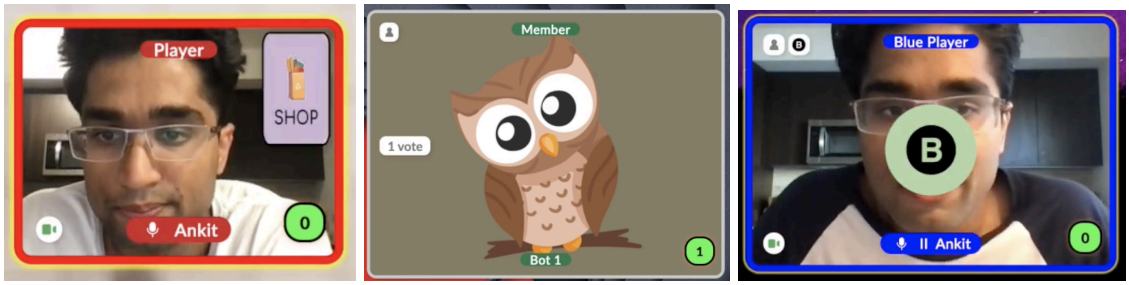
Inputs might be things like **actors** (who is doing the action), **targets** (who the action is being done to), and a **question** (what are the actors being asked to do).

CORE ACTIONS

The most common actions you will use over and over again include:

- createVote - actions that let the actors choose an answer to a multiple-choice question and/or vote at other players
- createInput - an action that lets the actors type in a free-form response and submit it
- createNotification - an action that shows a notification to the targets
- showRole - an action that shows the role of the actors to the targets
- updateScore - an action that updates the scores of certain players
- dealDeck, moveCards, and selectCentralWidgetDeck - core actions for card games
- emptyAction - a blank action that is useful for doing intermediate computations

YOUR VIDEO BOX



In a Ludio call, a player's video box can hold an *absurd amount of information*:

- Their name
- Their cam/mic status
    - Players can be orange-muted (they can unmute themselves), red-muted (they can't unmute themselves, the host/game must unmute them), or gray-muted (muted and dead in the game)
- Their role name ("Player", "Member", "Blue Player")
- Their team color (the red/green/blue background color of your name/role)
- A highlight around their video box (a red highlight usually means it's their turn)
- A player score (the number in the bottom-right)
    - You decide what the number means in your game
    - Scores are color-coded from yellow (worst) to green (best)
    - If you enable it, hovering over this icon shows you this player's cards
- A card (in the top-right - usually the last card played by that player)
- An "actor" icon (in the top-left - is this player an actor in the current action)
    - You can hover over this to see who this player voted at
- A "target" icon (in the middle-left - how many people are voting at this player)
    - An X means you can't vote at this player

    - You can hover over this to see who is voting at this player
- A multiple-choice option icon (e.g. B in the top-left of the third image + over your face)
    - Which multiple-choice option did this player vote for
- A conversation group (the “II” in Ankit’s name in the third image)
    - Which conversation group (i.e. private conversation) is this player a part of
    - You can hover over the II to see who else is in that conversation group

This information allows you to put information about other players *in their video boxes*, which is 1) more intuitive for players, who know exactly where to look for this information, and 2) less work for designers, who don’t have to make components to represent all this information.

Various actions let you set, hide, or edit all of these bits of information.

VARIABLES

To manage these inputs and outputs, Ludio uses **variables**, which are names for different bits of data. For example, you might have a variable called “numPlayers” which stores the number of players in the game. A variable might be a number (e.g. 5), a string (e.g. “Werewolf”), a boolean (true or false), a list of numbers/strings/booleans (e.g. [5, 6, 7]), or a dictionary of key-value pairs (e.g. {“Ankit”: “Werewolf”}).

Variables are classified into one of three types:

- **Preset** - you are setting the value on the fly
- **Cached** - you have saved the value of this variable before, you’re just referencing it
- **Computed** - you need to do some computation to get the value of this variable
    - Computation is done using **selectors**, which are functions that take inputs and return an output (e.g. to add two numbers, use the “add” selector”)

We encourage you to look through the templates and through the game jsons of other games to get a sense of how different games are put together. Action groups are a list of actions; an action is a payload of inputs and a result; each input is a variable; the result of an action can be saved to variables in the cache, which can then be used in future actions.

THE GAME JSON

A Ludio game json contains 9 sections:

- **gameInitOptions** - a set of parameters to initialize your game, such as the permitted player counts, the list of roles and teams, the composition of roles based on player count, and all your assets (images, audio, animations)
- **visualSettings** - extra visual configurations, mostly for card games
- **beforeLoopActions** - a list of actions that will get run exactly once as soon as the game starts, before the *gameLoop*
- **gameLoop** - a list of *action groups* that represent your core game logic

- **playersWinCondition** - logic to describe who wins (use this when the winner is usually one player)
- **winCondition** - logic to describe who wins (use this when the winner is usually a team)
- **postGameActions** - a list of actions that will get run when the game ends
- **turnSpectatorToPlayerActions** - a list of actions that will get run when a spectator wants to become a player in the current game
- **turnPlayerToSpectatorActions** - a list of actions that will get run when a player wants to leave the current game